

Simulation de trou noir de Schwarzschild

Physique relativiste et algorithme GPU

Ascari Yannick

6 juin 2026

Table des matières

1	Introduction	1
2	La métrique de Schwarzschild	2
2.1	Définition	2
2.2	Interprétation physique	2
3	Géodésiques nulles	2
3.1	Quantités conservées	2
3.2	Paramètre d'impact	2
3.3	Équation des orbites	3
3.4	Sphère de photons	3
4	Intégration numérique : méthode RK4	3
4.1	Système différentiel	3
4.2	Méthode de Runge-Kutta d'ordre 4	3
4.3	Conditions initiales	4
4.4	Conditions de terminaison	4
5	Modèle du disque d'accrétion	4
5.1	Géométrie	4
5.2	Profil de température	4
5.3	Effet Doppler relativiste	5
5.4	Redshift gravitationnel	5
6	Architecture GPU	5
6.1	Pipeline de rendu	5
6.2	Caméra orbitale	6
6.3	Complexité	6

1 Introduction

Ce document décrit les fondements physiques et algorithmiques d'une simulation temps-réel d'un trou noir de Schwarzschild, implémentée en Rust avec le pipeline GPU wgpu/WGSL.

L'objectif est de simuler le trajet de photons dans l'espace-temps courbé autour d'un trou noir, en intégrant numériquement les *géodésiques nulles* de la métrique de Schwarzschild. Le résultat est rendu pixel par pixel sur le GPU, chaque thread calculant une géodésique indépendante.

2 La métrique de Schwarzschild

2.1 Définition

La métrique de Schwarzschild décrit la géométrie de l'espace-temps autour d'une masse sphérique statique M . En coordonnées sphériques (t, r, θ, φ) , l'élément de ligne s'écrit :

$$ds^2 = - \left(1 - \frac{r_s}{r}\right) c^2 dt^2 + \frac{dr^2}{1 - r_s/r} + r^2 d\theta^2 + r^2 \sin^2 \theta d\varphi^2 \quad (1)$$

Où $r_s = \frac{2GM}{c^2}$ est le **rayon de Schwarzschild**. Dans la suite, on pose $c = G = 1$ et $r_s = 1$ (unités naturelles).

2.2 Interprétation physique

Le facteur $(1 - r_s/r)$ mesure la courbure locale de l'espace-temps :

- $r \gg r_s$: la métrique tend vers Minkowski (espace plat).
- $r = r_s$: l'horizon des événements – aucun signal ne peut s'échapper.
- $r < r_s$: les rôles de t et r s'échangent (singularité de coordonnées).

3 Géodésiques nulles

3.1 Quantités conservées

Un photon suit une **géodésique nulle** : $ds^2 = 0$. La symétrie sphérique de la métrique implique, via le théorème de Noether, deux quantités conservées le long de la géodésique :

$$E = \left(1 - \frac{r_s}{r}\right) \frac{dt}{d\lambda} \quad (\text{énergie}) \quad (2)$$

$$L = r^2 \frac{d\varphi}{d\lambda} \quad (\text{moment angulaire}) \quad (3)$$

Où λ est le paramètre affine le long de la géodésique.

3.2 Paramètre d'impact

Le **paramètre d'impact** $b = L/E$ est la distance perpendiculaire entre le trou noir et la trajectoire asymptotique du rayon :

$$b = |\vec{r}_0 \times \hat{d}| \quad (4)$$

Où $\vec{r}_0$ est la position d'origine et $\hat{d}$ la direction unitaire du rayon.

3.3 Équation des orbites

En posant $u = 1/r$ et en paramétrant par φ , la géodésique nulle se réduit à :

$$\boxed{\frac{d^2u}{d\varphi^2} + u = \frac{3}{2} r_s u^2} \quad (5)$$

Le terme $(-u)$ correspond à la géométrie plane – sans relativité, il donne une trajectoire rectiligne. Le terme $\frac{3}{2}r_s u^2$ est la **correction relativiste** qui courbe la lumière.

3.4 Sphère de photons

L'équation (5) admet une orbite circulaire instable à $r = \frac{3}{2}r_s$, la **sphère de photons**, correspondant au paramètre d'impact critique :

$$b_{\text{crit}} = \frac{3\sqrt{3}}{2} r_s \approx 2,598 r_s \quad (6)$$

Les rayons avec $b < b_{\text{crit}}$ sont absorbés. Ceux avec $b \approx b_{\text{crit}}$ orbitent plusieurs fois, formant les **anneaux de photons** visibles dans le rendu.

4 Intégration numérique : méthode RK4

4.1 Système différentiel

L'équation (5) est du second ordre. On la convertit en système du premier ordre en posant $v = du/d\varphi$:

$$\begin{cases} \frac{du}{d\varphi} = v \\ \frac{dv}{d\varphi} = -u + \frac{3}{2} r_s u^2 \end{cases} \quad (7)$$

4.2 Méthode de Runge-Kutta d'ordre 4

On intègre ce système avec un pas $\Delta\varphi = 0,01$ rad. Pour l'état (u_n, v_n) à l'étape n , on pose $f(u) = -u + \frac{3}{2}r_s u^2$:

$$k_1^u = v_n \quad k_1^v = f(u_n) \quad (8)$$

$$k_2^u = v_n + \frac{\Delta\varphi}{2} k_1^v \quad k_2^v = f(u_n + \frac{\Delta\varphi}{2} k_1^u) \quad (9)$$

$$k_3^u = v_n + \frac{\Delta\varphi}{2} k_2^v \quad k_3^v = f(u_n + \frac{\Delta\varphi}{2} k_2^u) \quad (10)$$

$$k_4^u = v_n + \Delta\varphi k_3^v \quad k_4^v = f(u_n + \Delta\varphi k_3^u) \quad (11)$$

La mise à jour de l'état :

$$\begin{pmatrix} u_{n+1} \\ v_{n+1} \end{pmatrix} = \begin{pmatrix} u_n \\ v_n \end{pmatrix} + \frac{\Delta\varphi}{6} \begin{pmatrix} k_1^u + 2k_2^u + 2k_3^u + k_4^u \\ k_1^v + 2k_2^v + 2k_3^v + k_4^v \end{pmatrix} \quad (12)$$

4.3 Conditions initiales

Pour un rayon d'origine $\vec{r}_0$ et de direction unitaire $\hat{d}$:

$$u_0 = \frac{1}{|\vec{r}_0|} \quad (13)$$

$$\left. \frac{du}{d\varphi} \right|_0 = -\frac{\hat{r}_0 \cdot \hat{d}}{b} \quad (14)$$

La position 3D à l'angle φ est reconstruite dans la base orthonormée du plan orbital $(\vec{e}_r, \vec{e}_\varphi)$:

$$\vec{r}(\varphi) = \frac{1}{u(\varphi)} [\cos \varphi \vec{e}_r + \sin \varphi \vec{e}_\varphi] \quad (15)$$

4.4 Conditions de terminaison

- $r \leq r_s$: absorption par l'horizon $\rightarrow$ pixel noir.
- $r \geq 60 r_s$: évaison vers l'infini $\rightarrow$ couleur du fond.
- $n > 500$ pas : rayon piégé près de la sphère de photons $\rightarrow$ pixel noir.

5 Modèle du disque d'accrétion

5.1 Géométrie

L'ancienne méthode (changement de signe y , équation $y_{n-1} \cdot y_n < 0$) est abandonnée au profit d'un disque volumétrique : à chaque pas RK4, le rayon accumule une contribution du disque pondérée par une gaussienne verticale :

$$H(r) = 0,03 \cdot r \quad (\text{hauteur d'échelle}) \quad (16)$$

$$w = \exp! \left(-\frac{y^2}{2H^2} \right) \cdot \Delta\varphi \cdot 10 \quad (17)$$

Cela modélise un disque d'épaisseur finie physiquement justifiée (disque mince $H/r \ll 1$), au lieu d'une intersection binaire avec un plan infiniment fin.

5.2 Profil de température

La température décroît avec le rayon. On paramétrise par :

$$t = 1 - \frac{r - r_{\text{inner}}}{r_{\text{outer}} - r_{\text{inner}}} \in [0, 1] \quad (18)$$

Le gradient de couleur va du blanc-bleu chaud ($t = 1$, près de l'ISCO) à l'orange-rouge froid ($t = 0$, bord extérieur).

Les bords du disque sont fondus via des fonctions *smoothstep* pour éviter les discontinuités :

$$\alpha = \text{smoothstep}(r_{\text{outer}}, r_{\text{outer}} - 2, r) \cdot \text{smoothstep}(r_{\text{inner}}, r_{\text{inner}} + 1,5, r) \quad (19)$$

5.3 Effet Doppler relativiste

La matière du disque suit une orbite képlérienne de vitesse :

$$v_{\text{orb}} = \sqrt{\frac{r_s}{2r}} \quad (20)$$

La direction orbitale au point d'intersection $\vec{p}$ est :

$$\hat{v}_{\text{orb}} = \frac{\vec{e}_y \times \hat{p}}{|\vec{e}_y \times \hat{p}|} \quad (21)$$

Le **facteur Doppler** modifie la fréquence et l'intensité perçues :

$$D = \frac{1}{1 - v_{\text{orb}} \cos \theta} \quad \text{où} \quad \cos \theta = \hat{v}_{\text{orb}} \cdot \hat{k} \quad (22)$$

$\hat{k}$ étant la direction du photon au point d'intersection. L'intensité spécifique observée scale en $D^{1,5}$ (**beaming relativiste**).

5.4 Redshift gravitationnel

Un photon émis à rayon r et observé à l'infini subit un décalage vers le rouge :

$$g = \sqrt{1 - \frac{r_s}{r}} \quad (23)$$

L'intensité finale combinée est :

$$\boxed{I_{\text{obs}} = D^{1,5} \cdot g \cdot I_{\text{base}}(r)} \quad (24)$$

L'intensité n'est plus calculée à un seul point d'intersection mais par **alpha-compositing** le long du rayon :

$$\text{rgb} += I_{\text{obs}} \cdot w \cdot (1 - \text{opacité}) \quad (25)$$

$$\text{opacité} += w \cdot (1 - \text{opacité}) \quad (26)$$

6 Architecture GPU

6.1 Pipeline de rendu

Chaque frame est produite en deux phases successives :

1. **Compute pass** – un thread GPU par pixel intègre une géodésique complète et écrit la couleur résultante dans une *storage texture*.
2. **Render pass** – un triangle plein écran lit la texture via un sampler et l'affiche sur la surface.

6.2 Caméra orbitale

La caméra est paramétrisée en coordonnées sphériques (θ, φ, r) :

$$\vec{r}_{\text{cam}} = r \begin{pmatrix} \sin \theta \cos \varphi \\ \cos \theta \\ \sin \theta \sin \varphi \end{pmatrix} \quad (27)$$

Sa position et sa base ($\hat{\text{right}}$, $\hat{\text{up}}$, $\hat{\text{forward}}$) sont transmises au compute shader via un *uniform buffer* mis à jour chaque frame. La direction de chaque rayon est construite depuis les coordonnées UV normalisées du pixel :

$$\hat{d} = \text{normalize} \left(u \cdot \hat{\text{right}} + v \cdot \hat{\text{up}} + \frac{\hat{\text{forward}}}{\tan(\alpha/2)} \right) \quad (28)$$

Où α est l'angle d'ouverture (FOV) de la caméra.

6.3 Complexité

Pour une résolution $W \times H$, on lance $\lceil W/8 \rceil \times \lceil H/8 \rceil$ workgroups de 8×8 threads. Chaque thread exécute au plus 500 itérations RK4, soit au pire $W \cdot H \cdot 500$ évaluations de $f(u)$ par frame, entièrement parallélisées sur le GPU.

Références

- [1] Misner, C.W., Thorne, K.S., Wheeler, J.A. (1973). *Gravitation*. W.H. Freeman.
- [2] Wald, R.M. (1984). *General relativity*. University of Chicago Press.
- [3] Lunin, J.P. (1979). Image of a spherical black hole with thin accretion disk. *Astronomy & Astrophysics*, 75, 228–235.
- [4] James, O. et al. (2015). Gravitational lensing by spinning black holes in astrophysics, and in the movie Interstellar. *Classical and Quantum Gravity*, 32(6).
- [5] wgpu – Safe and portable GPU abstraction in Rust. <https://wgpu.rs>